Amal9137



WinOS-Kernel / WinOS System

TABLE OF CONTENTS

Fault Control.....2
Base Initialization of WinOS-Kernel.....2
I. Displays the WinOS / OEM Logo.....2
II. Animates the TWP Throbber.....3
III. Initializes Essential Drivers.....3
IV. Starts LogonUI (ESP App).....3
WinOS Partition Layout (EFI).....3
Aggressive Data Security (ADS).....4
Windroid Security.....4

Fault Control

While developing, *we may run into many stupid issues and triple faults*. So, we need a **Death Screen** for **easier debugging**.



Base Initialization of WinOS-Kernel

After control transfers from **WinOS-EFI**, or the corresponding startup application on other platforms such as phones and ARM-based PCs, **WinOS System (WinOS-Kernel)** begins initialization.

The following sequence outlines the intended boot process. It may evolve as implementation progresses and additional functionality may be introduced.

I. Displays the WinOS / OEM Logo

WinOS System attempts to obtain the **OEM boot logo**, where supported. The logo is checked against the supported display dimensions and aspect-ratio limits. If the logo is unavailable, invalid, or unsuitable for the display, WinOS falls back to its own logo.

Where platform support and permissions allow, the obtained logo may be stored in the **EFI or equivalent boot partition** for subsequent use.

II. Animates the TWP Throbber

WinOS centres the selected logo on the screen and introduces the **TWP throbber**, used throughout *The Windroid Project*, including *Windroid Sim*.

The throbber animates while the remaining boot initialization takes place, providing a consistent visual identity across the project's components.

III. Initializes Essential Drivers

While the boot animation is running, WinOS loads the essential drivers required for the device from the **Common Programs** partition (see Pg. No. X).

Essential drivers may include:

- **Graphics Driver**
- **Audio Driver**
- **Network Driver**

Additional drivers are loaded as required by the platform and boot process.

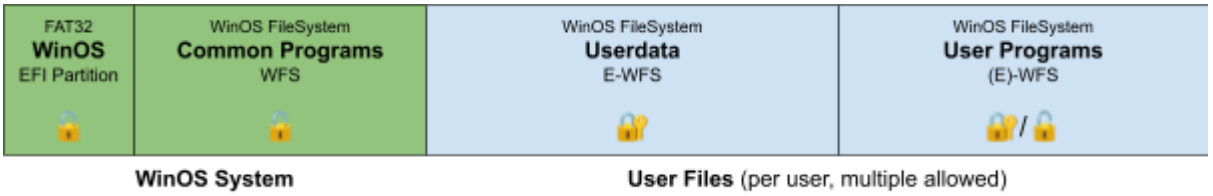
IV. Starts LogonUI *(ESP App)*

WinOS discovers available user partitions across the installed disks and lists the accessible user installations in **LogonUI**. The user can select an installation, enter their password where required, and proceed to the desktop.

- **Authentication Protection: Windroid Security** maintains a *global record* of failed authentication attempts. When repeated failures occur within a defined time window, it applies progressively increasing restrictions to slow further attempts. If the activity reaches a high-confidence threshold for automated authentication spam, WinOS may force a **shutdown as a last-resort protective response**. The attempt record persists across restarts and shutdowns.
- **Passwordless Accounts:** Accounts may operate without passwords; however, their **Userdata will not be encrypted**.
- **Ephemeral Sessions:** Users may enter an *ephemeral session* without accessing their Userdata, allowing them to use WinOS without unlocking their personal data.

If no usable User Installation is available, **WinOS** proceeds to *WinOS Setup*, where the user can configure a new installation.

WinOS Partition Layout (EFI)



WinOS - Contains all the ESP apps (like LogonUI, Settings etc., system apps, the apps WinOS requires for UX). **Not Encrypted**.

Common Programs - Contains programs which WinOS doesn't really 'require' to run, like drivers. **Not Encrypted**.

Userdata - Contains the appdata of a user. ADS Compatible. **Encrypted** *(or not, depending on password lock)*.

User Programs - Contains all user programs. ADS Compatible. **Optional Encryption**.

Aggressive Data Security (ADS)

When toggled ON, WFS *encrypts every single file after every single use*. **May cause huge performance issues and app incompatibility**.

Windroid Security

Windroid Security is the security subsystem of **WinOS.** Unlike other OSes, all the settings, allowances etc. are **set by the user**, fulfilling the User is king policy. It is not strictly the OS' kernel, but rather a **vital part of it.**

It impacts various areas such as;

- **LogonUI; Windroid Security** strictly monitors the user's attempt(s) to login to the user desktop. If it sees spam attempts on the decrypting stage (not when the user is already logged in). **Windroid Security force shutdowns the whole system.**

  In normal cases, when the user partition is *already decrypted*, the user is unallowed to reattempt for $(x(cur)/x)^2$ seconds **(exponential)**, where

  x(cur) = current attempt count

  x = user set attempt limit until timeout